CPT204 2023-2024 F S2 - Suggested Answers

Short Answer Questions

1. primitive; reference
2. overriding; overloading
3. false and false
4. interface; concrete methods / instance variables / constructors
5. [New York, Dallas]
6. Line 4
7. generic; compiler
8. <E> void print(E[] list)
9. ArrayList; LinkedList
10. list1.remove(list2) removes the object list2 if present. Since list2 is not an element of list1, list1 remains [red, yellow, green].
11. Stacks; Queues
12. 10 20 30 40 50 60
13. [ABC, FGH]
14. HashSet<Integer>()
15. HashMap
16. Big O notation
17. O(n); O(log n)
18. O(n)
19. O(n^2)
20. [9, 12, 4, 15, 20]
21. 3 possible integer values: 10, 11, 12. If strict uniqueness is required, 11 and 12 only.
22. parallel edges; complete graph
23. DFS; BFS
24. Prim's algorithm; Dijkstra's algorithm
25. O(log n); O(log n)
26. preorder; postorder
27. left-heavy; right-heavy
28. Rotation depends on the figure in the original image. Use AVL insertion rule: LL -> right rotation; RR -> left rotation; LR -> left-right rotation; RL -> right-left rotation.
29. open addressing; separate chaining
30. linear; quadratic

Coding Question 1 - RealEstateAnalyzer

```
import java.util.ArrayList;
import java.util.Arrays;

public class RealEstateAnalyzer {
    public static void main(String[] args) {
        ArrayList<Double> prices2 = new ArrayList<>();
        System.out.println(max(prices2));

        ArrayList<Double> prices1 = new ArrayList<>(Arrays.asList(
                1250000.1, 1750000.3, 2100000.6, 1900000.8, 2300000.2));
        System.out.println(max(prices1));
    }

    public static <E extends Comparable<E>> E max(ArrayList<E> list) {
        if (list == null || list.isEmpty()) {
            return null;
        }
        E highestPrice = list.get(0);
        for (int i = 1; i < list.size(); i++) {
            if (list.get(i).compareTo(highestPrice) > 0) {
                highestPrice = list.get(i);
```

```
            }
        }
        return highestPrice;
    }
}
```

Coding Question 2 - FlightBoardingSystem

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.PriorityQueue;
import java.util.Set;

public class FlightBoardingSystem {
    public static void main(String[] args) {
        PriorityQueue<String> firstClassQueue = new PriorityQueue<>(
                Arrays.asList("Alice", "Bob", "Charlie", "Diana", "Ethan", "Fion
a"));
        PriorityQueue<String> businessClassQueue = new PriorityQueue<>(
                Arrays.asList("Charlie", "Diana", "Grace", "Ian"));
        PriorityQueue<String> economyClassQueue = new PriorityQueue<>(
                Arrays.asList("Alice", "Grace", "Ethan", "Hannah", "Ian"));

        System.out.println(findExclusiveFirstClassPassengers(
                firstClassQueue, businessClassQueue, economyClassQueue));
    }

    public static Set<String> findExclusiveFirstClassPassengers(
            PriorityQueue<String> firstClassQueue,
            PriorityQueue<String> businessClassQueue,
            PriorityQueue<String> economyClassQueue) {
        Set<String> exclusiveFirstClass = new HashSet<>(firstClassQueue);
        exclusiveFirstClass.removeAll(businessClassQueue);
        exclusiveFirstClass.removeAll(economyClassQueue);
        return exclusiveFirstClass;
    }
}
```

Expected exclusive first-class passengers: Bob, Fiona.